

Возможности и резервный маршрут

Основной план, модуль 01

Краткое содержание

Резервный маршрут — это не простое переключение на следующую языковую модель в списке, а повторный выбор только среди вариантов, которые сохраняют смысл текущего шага агента. Маршрутизатор сначала проверяет доступные инструменты, строгую схему, типы входных данных, размер контекста, потоковую выдачу и явно включаемые экспериментальные возможности. Лишь после этого совместимые маршруты ранжируются по работоспособности, задержке и стоимости. Если ни один вариант не подходит, корректный результат — ошибка `NoCompatibleRoute`, а не молчаливое удаление инструмента или ослабление схемы.

Ключевые понятия

Понятие	Смысл
Маршрут	<code>ModelRoute</code> : устойчивый идентификатор, адаптер и конкретный профиль возможностей
Профиль возможностей	<code>CapabilityProfile</code> : проверенные свойства конкретной конфигурации поставщика
Требования	<code>CapabilityRequirements</code> : обязательные условия одного решения модели, сформированные до выбора маршрута
Обязательное требование	Условие корректности, которое нельзя ослабить ради доступности
Критерий предпочтения	Работоспособность, задержка, стоимость или оценка качества среди уже совместимых маршрутов
Отчёт о совместимости	<code>CompatibilityReport</code> : результат проверки со всеми причинами отказа и источником сведений
Нет подходящего маршрута	<code>NoCompatibleRoute</code> : отказ до вызова модели, если ни один маршрут не удовлетворяет требованиям
Явное ослабление	Новое решение приложения с изменёнными требованиями и записью аудита

Возможности относятся не только к `model_name`, а ко всей фактически используемой конфигурации:

поставщик + версия модели + вариант API + параметры/заголовки + регион/учётная запись

Псевдоним модели со временем может начать указывать на другую версию. Одна и та же модель может иметь разные возможности в OpenAI Responses и OpenAI Realtime или устаревшей точке API. Заголовок бета-функции, права учётной записи и регион тоже меняют фактический контракт.

Основные идеи

Проблема: доступность без подмены смысла

Пусть агент должен разобрать снимок экрана, вернуть результат по строгой схеме `change_plan/v1` и при необходимости предложить `propose_patch`. Основной маршрут недоступен. Дешёвый резервный вариант принимает только текст, генерирует JSON без гарантии соответствия схеме и не знает `propose_patch`.

Если маршрутизатор удалит изображение, инструмент или строгую схему и всё же вернёт обычный `ModelResponse`, агент продолжит работу в ложном состоянии. Вызов модели снаружи выглядит успешным, хотя пространство допустимых решений уже изменилось. Это не просто увеличение задержки или снижение качества, а подмена задачи.

Инвариант маршрутизации:

возможности(выбранный_маршрут) удовлетворяют всем обязательным требованиям

Состояние `unknown` («неизвестно») не равносильно `supported` («поддерживается»). Если метаданных недостаточно, маршрут считается несовместимым до успешного контрольного запроса или обновления описания возможностей.

Требования и профили возможностей — разные модели данных

`CapabilityProfile` описывает, что конкретный маршрут технически способен предложить. `CapabilityRequirements` описывает, что требуется конкретному переходу состояния агента. Один маршрут может подходить для текстовой классификации и не подходить для шага с изображением и инструментом.

Минимальное отношение совместимости:

```

requirements.tools ⊆ profile.available_tools
requirements.input_modalities ⊆ profile.input_modalities
requirements.output_modalities ⊆ profile.output_modalities
requirements.min_input_tokens ≤ profile.max_input_tokens
requirements.min_output_tokens ≤ profile.max_output_tokens
если profile.max_total_tokens задан:
    requirements.min_input_tokens + requirements.min_output_tokens
        ≤ profile.max_total_tokens
requirements.schema.mode ∈ profile.schema_modes
schema_preflight(requirements.schema, profile.schema_dialect) = compatible
requirements.stream_mode ∈ profile.stream_modes
requirements.experimental ⊆ profile.experimental_features
profile.maturity ∈ requirements.allowed_maturity

```

Входной бюджет включает системную инструкцию, историю, определения инструментов и результаты их вызовов, а также входные данные разных типов. Резерв ответа проверяется отдельно. Если поставщик дополнительно задаёт общее окно, проверяется и сумма входного бюджета с резервом ответа. Большое входное окно само по себе не гарантирует, что модель сможет вернуть ответ нужной длины. Одного `len(user_text)` недостаточно. Типы входных и выходных данных хранятся раздельно: приём изображения не означает умение создать изображение.

Метка `strict` — необходимое, но не достаточное условие. Поставщики поддерживают разные подмножества и диалекты JSON Schema. Поэтому реестр хранит диалект и ограничения, а схема проходит предварительную проверку для конкретного маршрута. Неизвестное ключевое слово или неподдержанная комбинация дают типизированную причину отказа до вызова модели.

Поддержка инструментов тоже не сводится к `supports_tools=True`. Нужны как минимум точные идентификаторы инструментов, место выполнения (`client` — на стороне клиента, `server` — на стороне поставщика), режим схемы и ограничения параллельных вызовов. Наличие инструмента в профиле означает лишь способность модели предложить вызов; полномочий на выполнение оно не выдаёт.

Независимые признаки могут вводить в заблуждение

Поддержка инструментов и строгого структурированного ответа по отдельности не доказывает, что их можно использовать вместе. Реальный профиль должен хранить поддерживаемые режимы и ограничения:

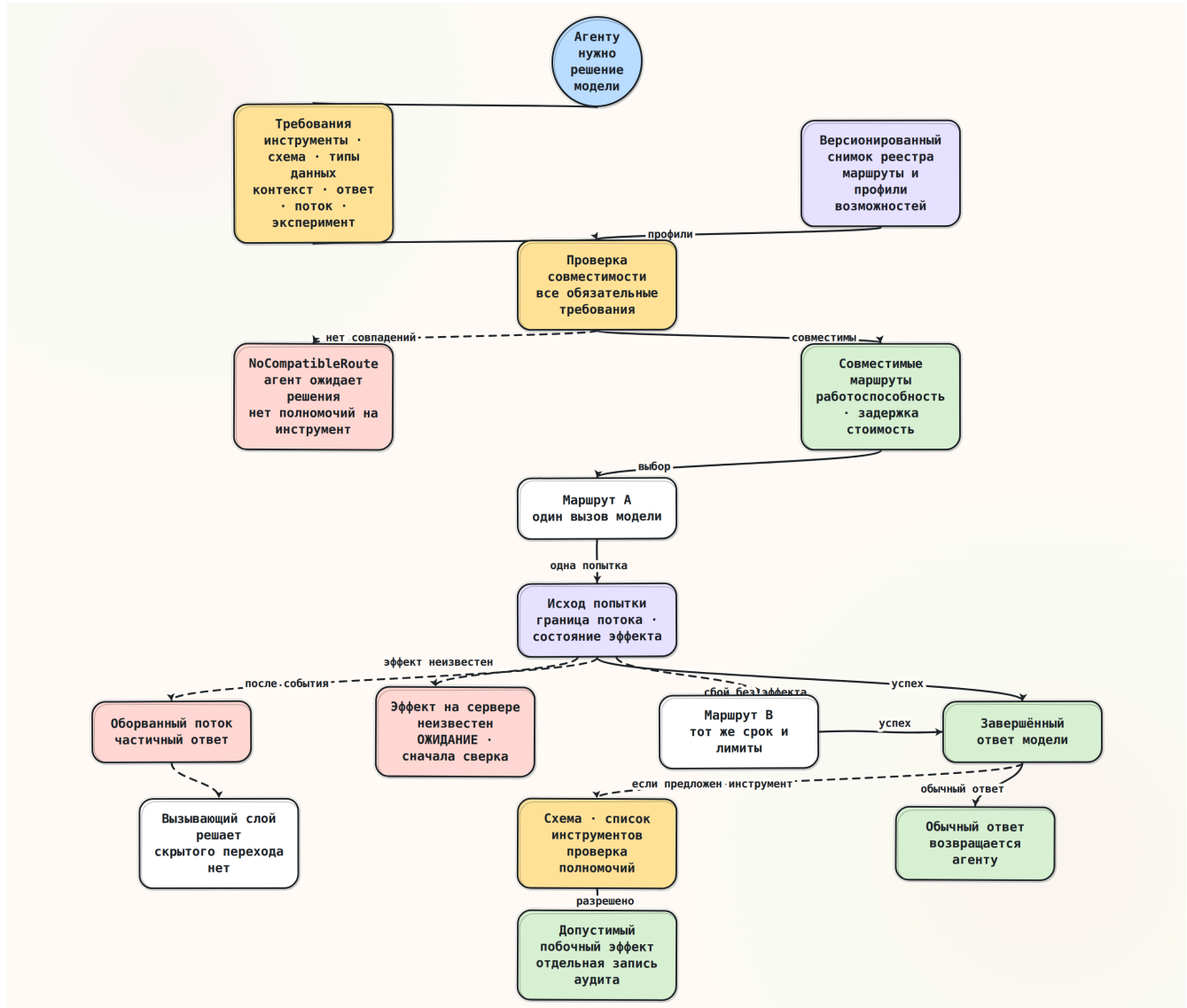
```

mode: streaming + client_tools + strict_tool_schema
mode: non_streaming + structured_final_response
constraint: structured_final_response + built_in_tools = preview_only

```

В практической части темы 1.4 начнём с явных полей и отдельного списка версионированных экспериментальных возможностей. Ограничение такой модели нужно зафиксировать: промышленный реестр обязан описывать существенные комбинации, а не вычислять их как произведение независимых булевых признаков.

Поток выполнения



1. Состояние агента и разрешённый набор инструментов порождают обязательные требования.
2. Маршрутизатор читает неизменяемый снимок реестра и строит `CompatibilityReport` для каждого возможного маршрута.
3. Проверка совместимости исключает варианты с любым несоответствием до вызова адаптера.
4. Политика ранжирования сравнивает только совместимые маршруты по работоспособности, задержке, стоимости или оценке качества.
5. Выбранный адаптер выполняет ровно один запрос к поставщику. Маршрутизация остаётся над `ModelClient`, а не скрывается внутри него.

6. Вызывающий слой может выбрать следующий совместимый маршрут в пределах того же общего крайнего срока и тех же лимитов только при сбое, допускающем переключение: до первого принятого события потока **и** когда побочный эффект инструмента на стороне сервера заведомо невозможен либо сверка уже подтвердила его отсутствие.
7. После принятого события действует правило темы 1.3: нельзя неявно переключаться на другую модель и склеивать части разных попыток. Наружу идёт `ModelStreamInterrupted`, а дальнейшее решение принимает вызывающий слой.
8. Завершённый `ModelResponse` возвращается агенту. Предложение вызвать инструмент ещё проходит проверку схемы, списка разрешённых инструментов и полномочий до побочного эффекта.

Отсутствие первого события известно лишь со стороны клиента и не доказывает, что инструмент на стороне поставщика не успел выполниться. Если состояние побочного эффекта неизвестно, агент переходит в состояние ожидания сверки; автоматическое переключение запрещено. Ключ идемпотентности помогает сверить или безопасно повторить уже известную операцию, но сам по себе не доказывает эквивалентность нового решения модели.

Границы ответственности

Компонент	Владеет	Не должен делать
Реестр	Профили, источник сведений, версия реестра, <code>verified_at</code>	Ранжировать, вызывать модель или разрешать выполнение инструмента
Проверка совместимости	Сопоставление обязательных требований и точные причины отказа	Ослаблять требования ради доступности
Политика ранжирования	Порядок уже совместимых маршрутов	Делать несовместимый маршрут допустимым
Адаптер / <code>ModelClient</code>	Один запрос к поставщику и нормализация ответа	Скрыто менять модель или выбирать резервный маршрут
Агент / приложение	Явное ослабление требований и переход состояния	Считать выбор маршрута полномочием на вызов инструмента
Граница выполнения инструмента	Схема, список разрешений, полномочия, идемпотентность	Исполнять предложение из незавершённого ответа или непроверенное предложение

Маршрутизатор не изменяет шаблон запроса, независимый от маршрута. Поле `ModelRequest.model` обязательно, поэтому после выбора строится новый `ModelRequest(model=selected_route.model, ...)`. Значения `messages`, `tools`, `response_format` и идентификатор запроса приложения остаются прежними. Резервный маршрут В никогда не получает идентификатор модели от маршрута А. Если для переключения нужно удалить инструмент, сжать контекст, заменить изображение описанием или перейти от строгой схемы к обычному тексту, это уже новое решение приложения, новый запрос и отдельная запись аудита.

Причинная связь с состоянием агента

Несовместимая ветка:

проверка совместимости отклоняет дешёвый резервный маршрут
→ агент: BUILDING_CHANGE_PLAN → BLOCKED_BY_CAPABILITY
→ завершённого решения модели нет
→ предложения вызвать инструмент и полномочий на выполнение нет
→ состояние репозитория не меняется

Совместимая ветка с предложением вызвать инструмент:

совместимый резервный маршрут → завершённый ModelResponse
→ предложение модели входит в состояние агента
→ проверка схемы, списка инструментов и полномочий
→ только затем допустимый побочный эффект инструмента

Если завершённый ответ не содержит предложения вызвать инструмент, он просто входит в состояние агента; ветки проверки полномочий и побочного эффекта нет.

Маршрутизация сохраняет возможность получить корректно сформированное предложение модели. Она не разрешает побочный эффект и не доказывает качество ответа.

Примеры

Матрица совместимости

Текущий шаг требует `search_evidence`, `propose_patch`, изображение на входе, строгую схему `change_plan/v1` в совместимом диалекте, потоковую выдачу, входной контекст объёмом `52k`, резерв ответа `4k` и версионированную возможность `reasoning_budget:v1`.

Требование	Основной А	Резервный В	Дешёвый С
Инструменты {search_evidence, propose_patch}	оба	оба	только search_evidence
Строгая схема change_plan/v1	да	да	JSON без гарантии схемы
Вход {text, image}	да	да	только текст
Входной контекст 52k	128k	64k	128k
Резерв ответа 4k	до 16k	до 8k	до 16k
Потоковая выдача	да	да	да
reasoning_budget:v1	да	да	нет
Результат	совместим	совместим	четыре несоответствия

Если А возвращает 503 до первого события и его контракт исключает побочный эффект на стороне сервера, политика выбирает В. Низкая цена не делает С допустимым. Если В отсутствует, маршрутизатор возвращает NoCompatibleRoute с четырьмя причинами и не вызывает несовместимый клиент. Если состояние побочного эффекта после попытки А неизвестно, агент ожидает сверки вместо автоматического перехода к В.

Небольшой набросок, независимый от поставщика

Это форма будущего контракта, а не реализация практики:

```

@dataclass(frozen=True, slots=True)
class SchemaRequirements:
    mode: str
    dialect: str
    keywords: frozenset[str]

@dataclass(frozen=True, slots=True)
class CapabilityRequirements:
    tools: frozenset[str]
    input_modalities: frozenset[str]
    output_modalities: frozenset[str]
    schema: SchemaRequirements
    min_input_tokens: int
    min_output_tokens: int
    stream_mode: str
    experimental: frozenset[str]
    allowed_maturity: frozenset[str]

def select(routes, required):
    checks = tuple(check(route.profile, required) for route in routes)
    compatible = [
        route
        for route, report in zip(routes, checks, strict=True)
        if not report.gaps
    ]
    if not compatible:
        raise NoCompatibleRoute(checks)
    return rank(compatible)[0]

```

Важен порядок операций: `check` → `filter` → `rank`. Конструкция `rank(all_routes)[0]` → `strip_unsupported_fields` принципиально неверна.

Журнал выбора маршрута

```

requirements_revision=change-plan/v1
registry_revision=2026-08-12.3
candidate=primary-a      compatible=true
candidate=fallback-b     compatible=true
candidate=cheap-c        gaps=tool,strict_schema,image,experimental
selected=fallback-b      reason=primary_effect_free_failure_before_first_event

```


Журнал нужен для разбора инцидента и воспроизведения решения. Он фиксирует факты выбора, но не обязан содержать текст исходного запроса или конфиденциальные аргументы инструмента.

Заметки и наблюдения

Связь с текущим кодом

- `ModelRequest.tools` и `JsonSchemaFormat` уже выражают часть требований.
- `InputMessage.content` пока содержит только `str`; типы входных и выходных данных отдельно не моделируются.
- `ModelClient.complete()` и `stream()` означают один запрос к поставщику. Наличие `stream()` в `Protocol` не доказывает поддержку потоковой выдачи конкретным маршрутом.
- `UnsupportedCapability` защищает границу адаптера, когда поставщик фактически отклоняет запрос. Для отказа по нескольким маршрутам дополнительно нужен `NoCompatibleRoute` с отчётами по всем кандидатам.
- `StreamingRuntime` темы 1.3 ограниченно повторяет запрос к тому же клиенту после раннего 429; это не переключение между моделями. Новый маршрутизатор должен наследовать исходный общий крайний срок.

Новые типы возможностей лучше добавлять в отдельный урок, не меняя поля `ModelRequest`: текущие темы и контрактные тесты зависят от него. До выбора маршрута нужен независимый шаблон; затем для каждой попытки строится новый `ModelRequest` с идентификатором модели выбранного маршрута и неизменными `messages`, `tools`, `response_format` и идентификатором запроса приложения.

Реестр — проверенное описание, а не вечная истина

Способы получения сведений у поставщиков различаются. OpenAI Models API отдаёт базовые идентификационные сведения о моделях, а сравнение возможностей опубликовано в отдельном каталоге. Anthropic Models API включает больше метаданных возможностей. Gemini Models API публикует ограничения по токенам и доступные способы генерации, но не весь контракт маршрутизации. Поэтому промышленный реестр объединяет версионированное описание, первичную документацию, настройки учётной записи и региона, а также контролируемые проверки.

Профиль должен хранить `source`, `observed_at`, используемый вариант API, `snapshot`, `alias`, регион и права учётной записи. Устаревший профиль всё равно может ошибиться, поэтому адаптер сохраняет защитную ошибку `UnsupportedCapability` и обновляет телеметрию.

Чего совместимость не доказывает

- качество, фактическую и смысловую корректность результата — это проверяет оценка качества;
- доступность прямо сейчас зависит от текущей работоспособности сервиса и доступной квоты;

- полномочия учётной записи и размещение данных проверяет политика доступа;
- корректность конкретного предложения вызвать инструмент проверяют схема и бизнес-правила;
- право выполнить побочный эффект — это отдельная граница полномочий.

Эти факторы участвуют в допуске маршрута или его ранжировании, но не должны смешиваться в один непрозрачный признак `supports_request=True`.

Ограничения будущей практики

- Начальный реестр будет статическим и детерминированным, без обращения к реальному API поставщика.
- Поля контекста будут задавать минимальные входной бюджет и резерв ответа, а не заменять полноценный токенизатор и точную оценку размера входного запроса.
- Идентификация инструмента начнётся с имени; версия и отпечаток схемы появятся в модуле про протокол инструментов.
- Простая модель сочетаний возможностей не покрывает все ограничения поставщиков.
- Тесты свойств докажут инварианты маршрутизации на искусственных профилях, но не реальные права учётной записи и не качество модели.

Вопросы для повторения

1. Почему резервный маршрут меняет смысл решения, а не является обычной повторной попыткой?
2. Чем обязательное требование отличается от критерия предпочтения?
3. Почему `supports_tools=True` недостаточно для агента, использующего инструменты?
4. Почему приём и создание изображения — разные возможности?
5. Зачем вместе проверять размер контекста и резерв на ответ?
6. Почему поддержку инструментов и строгой схемы нельзя всегда проверять независимо?
7. Что должен сделать маршрутизатор, если метаданные имеют состояние `unknown`?
8. Почему выбор маршрута не выдаёт полномочий на выполнение инструмента?
9. Почему отсутствие события потока недостаточно и когда переключение между моделями всё же допустимо?
10. Какие данные нужны в отчёте о совместимости и журнале выбора?

Итоги

Маршрутизация с учётом возможностей отделяет доступность от подмены смысла. Реестр хранит версионированные факты, проверка совместимости отказывает при неопределённости, политика ранжирования выбирает только среди подходящих вариантов, а адаптер выполняет один запрос к поставщику. Несовместимый резервный маршрут не становится допустимым из-за низкой цены или хорошей работоспособности. Любое ослабление требований — отдельное решение приложения с новым состоянием агента и записью аудита.

После команды `к практике` реализуем механизм с нуля: неизменяемый реестр без `if model_name`, типизированные отчёты о совместимости, маршрутизатор с отказом при неопределённости и детерминированные тесты свойств. Фреймворк на этой стадии не нужен.

Источники

Возможности проверены по первичной документации `2026-08-12`: OpenAI Responses по спецификации OpenAPI `2.3.0`, Anthropic API с обязательным `anthropic-version: 2023-06-01`, Gemini API `v1` и клиентской библиотекой, которая по умолчанию обращается к `v1beta`. Ссылки фиксируют источники выводов, а псевдонимы моделей считаются изменяемыми.

OpenAI: функции, ответы, поток, сравнение, модели, ответы API. Anthropic: инструменты, строгий режим, аргументы, модели, бета-функции, версия API. Google Gemini: функции, ответы, модели, API, версии моделей, версии API. Локально: план, контракт, среда выполнения.